

Esame OS Internals (Cabodi)

Data: 04/09/2023

Nota Generale: In tutte le domande, ogni volta che il risultato è una selezione, un SI/NO, o un numero, sono necessari una motivazione/spiegazione o passaggi intermedi che portino al risultato.

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Paginazione, Working Set e Page Fault)

Contesto: Si consideri il seguente frammento di programma:

```
#define N 32
data_t M[2*N][N];
int i, j, k;

for (k = 0; k <= 4; k++) {
    for (i = 0; i < 2*N; i++) {
        for (j = k; j < N; j += 4) {
            M[i][j].diag = i - j;
        }
    }
}
```

Il codice macchina generato dal programma viene eseguito su un sistema con gestione della memoria basata su paginazione a richiesta, pagine da 2KB, sostituzione delle pagine guidata da una politica Working set (versione esatta), con $\delta = 10$. Si sa che:

1. `sizeof(data_t) = 1024`
2. `data_t` ha un campo `diag` di tipo `int` (dimensione 32 bit)
3. il segmento di codice (istruzioni macchina) ha dimensioni inferiori a una pagina
4. `M` viene allocato a partire dall'indirizzo logico `0x5820C800`
5. la matrice `M` viene allocata seguendo la strategia "row major", cioè per righe (prima riga, seguita da seconda riga, ...).

Rispondere alle seguenti domande:

- A) Quante pagine (e frame) sono necessarie per memorizzare la matrice?
- A) Supponendo che le variabili `i`, `j` e `k` siano allocate nei registri (l'accesso ad esse non produce riferimenti di memoria), quanti riferimenti totali $NT = NW + NR$ (NR per la lettura e NW per la scrittura dei dati) produce il programma proposto (non considerare i fetch delle istruzioni)?
- A) Calcolare il numero di page fault generati dal programma proposto (motivare/giustificare la risposta).

- A) Completando correttamente il seguente programma, è possibile fornire un codice equivalente (in termini di comportamento) al precedente, che produce meno page fault. Completare il programma e calcolare quanti page fault vengono generati (motivare la risposta).

```
#define N 32
data_t M[2*N][N], *V;
int k;

V = &(M[0][0]);
for (k = 0; k < 2*N*N; k++) {
    V[k].diag = /* completare qui */;
}
```

Domanda 2 (File System Unix-like e Inode)

Contesto: Si consideri un file system Unix-like, basato su inode, con 15 puntatori/indici (12 diretti, 1 singolo indiretto, 1 doppio indiretto e 1 triplo indiretto). I puntatori/indici hanno dimensione 32 bit e i blocchi su disco hanno dimensioni 4KB. Il file system risiede su una partizione del disco, dove 2 TB sono riservati ai blocchi di dati. Lo spazio aggiuntivo riservato ai metadati (inclusi i blocchi di indice) può essere trascurato ai fini di questo esercizio.

- A) È noto che il file più piccolo nel File System ha dimensioni 8MB e il file più grande ha dimensioni 8GB. Si utilizzi N_2 per il numero di file con indice indiretto doppio e N_3 per il numero di file con indiretto triplo. Calcolare i valori massimi (i limiti superiori) per N_2 e N_3 .
- A) Dato un file binario di dimensioni 20490,5 KB, calcolare esattamente quanti blocchi di indice e di dato usa il file. Calcolare anche la frammentazione interna (per i blocchi di dato).
- A) Si consideri lo stesso file della domanda B. Si supponga di effettuare la chiamata `lseek(fd, offset, SEEK_END)` per posizionare l'offset del file (`SEEK_END` significa che l'offset viene riferito alla fine del file) per la successiva operazione di lettura/scrittura. Supponiamo che `fd` sia il descrittore del file associato al file (già aperto) e che `offset = -220`. Calcolare il numero di blocco logico (relativo ai blocchi logici del file, numerati a partire da 0) in corrispondenza del quale viene spostato l'offset.
-

Domanda 3 (Prestazioni e Allocazione File)

- A) Un file di testo di grandi dimensioni contiene N righe di lunghezza fissa (50 caratteri per ogni riga). Occorre ordinare le righe nel file usando un algoritmo di ordinamento di complessità lineartmica ($O(N \log N)$). A causa delle dimensioni, l'ordinamento viene implementato direttamente sul file (sfruttando memory mapping del file oppure utilizzando in modo opportuno le primitive di posizionamento, lettura e scrittura su file). A seconda del file system sottostante (strategia di allocazione dei file), quale complessità si prevede per l'ordinamento? (Scegliere tra $O(N)$, $O(N \log N)$, $O(N^2)$, $O(N^2 \log N)$, $O(N^3)$ per allocazione Contigua, Linked list, FAT, Inodes, e fornire una breve motivazione).

- A)** Un grande insieme di dati viene memorizzato su file utilizzando la strategia denominata "indice e file relativo" ("index and relative file"): il file indice contiene una sequenza di N record di dimensione fissa, ciascuno contenente una chiave di ricerca e un puntatore (un indirizzo logico) al record corrispondente nel file relativo. Il file di indice viene ordinato in base a chiavi di valore crescente. Il file relativo contiene N record di lunghezza variabile. I record NON sono ordinati. A seconda del file system sottostante (strategia di allocazione dei file), quale complessità si prevede per l'operazione di stampa di tutti i dati, seguendo l'ordine delle chiavi? (Scegliere tra $O(\log N)$, $O(N)$, $O(N \log N)$, $O(N^2)$ per allocazione Contigua, Linked list, FAT, Inodes, e fornire una breve motivazione).

2 SECTION 2: OS161

Domanda 4 (Sincronizzazione OS161 - Producer/Consumer)

Contesto: Si considerino tre thread del kernel OS161 che implementano un'attività di trasferimento dati basata su un modello producer/consumer (2 producer, 1 consumer). I thread condividono un buffer di dati, implementato come struttura C di tipo `struct prodConsBuf`, definito come segue:

```
struct prodConsBuf {
    data_t data;
    int dataReady;
    struct lock *pc_lk;
    struct cv *pc_cv;
};
```

Tutte le operazioni (scrittura/lettura), necessitano di mutua esclusione. Ogni volta che nel buffer non sono presenti dati (quindi il buffer è vuoto), il flag `dataReady` è 0. Dopo che un produttore scrive nuovi dati nel buffer (campo `data`), pone `dataReady` al valore 1 (quindi buffer pieno) e segnala il consumer (il lavoro viene eseguito chiamando la funzione `producerWrite`). Il consumer esegue in modo iterativo la funzione `consumerRead`.

Rispondere alle seguenti domande:

- La struttura condivisa può essere posizionata nello stack di thread o deve essere una variabile globale o altro?
- Poiché i campi `pc_lk` e `pc_cv` sono puntatori, dove dovrebbero essere chiamate le funzioni `lock_create()` e `cv_create()`? Nel thread producer, nel consumer thread? Altrove?
- Le funzioni di produttore e consumatore sono parzialmente realizzate e riportate di seguito. Completare la loro implementazione dove indicato (<1>, <2> e <3>). Si noti che occorre evitare "busy waiting", quindi un produttore segnala al consumatore quando i dati sono pronti. Poiché un produttore potrebbe dover attendere che il consumatore legga ("consumi") i dati precedenti, prima che il produttore sia autorizzato a scrivere un nuovo dato, anche il consumatore deve fare una segnalazione.

```
/* error checking/handling instructions are omitted for simplicity */
void producerWrite (struct prodConsBuf *pc, data_t *dp) {
    /* lock for mutual exclusion */
    lock_acquire (pc->pc_lk);

    /* properly handle here the fact that the buffer could be full
       (filled by the other producer and not yet read by the consumer) */
    <1>

    /* copy data: done here by simple assignment
       a proper function could be used instead */
    pc->data = *dp;
    pc->dataReady = 1; /* set flag */

    /* signal the consumer */
    <2>

    /* just release the lock and return */
```

```

    lock_release (pc->pc_lk);
}

void consumerRead (struct prodConsBuf *pc, data_t *dp) {
    /* lock for mutual exclusion */
    lock_acquire (pc->pc_lk);

    while (!pc->dataReady) {
        cv_wait(pc->pc_cv, pc->pc_lk);
    }

    /* copy data: done here by simple assignment
       a proper function could be used instead */
    *dp = pc->data;
    pc->dataReady = 0; /* reset set flag */

    /* signal a producer waiting, if any */
    <3>

    /* release the lock and return */
    lock_release (pc->pc_lk);
}

```

Domanda 5 (System Calls: open, close, write in OS161)

Si consideri una possibile implementazione delle chiamate di sistema `open()` e `close()` in OS161.

B) La "per-process file table" è allocata in memoria user o in memoria kernel?

B) Si supponga che due processi utente, in esecuzione contemporaneamente, chiamino `write(fd, v, N)`, dove `fd` è 5 per entrambi, ci si aspetta che (per ogni opzione rispondere SI/NO e motivare):

1. I processi scrivano sullo stesso file.
2. Non scrivano mai sullo stesso file.
3. Generalmente scrivano su file diversi ma a volte potrebbero scrivere sullo stesso.
4. Per scrivere sullo stesso file, i due processi dovrebbero utilizzare valori diversi per `fd`.

B) Data la seguente implementazione di `sys_write`:

```

int sys_write (int fd, userptr_t buf_ptr, size_t size) {
    int i;
    char *p = (char *)buf_ptr;

    if (fd != STDOUT_FILENO && fd != STDERR_FILENO) {
        return file_write(fd, buf_ptr, size);
    }

    for (i = 0; i < (int)size; i++) {
        putchar(p[i]);
    }
}

```

```
    }  
    return (int)size;  
}
```

Supponendo che standard input, output ed error siano mappati alla console (quindi non è possibile alcun loro reindirizzamento a file), si dica se è possibile sostituire la gestione di `stdout/stderr` mediante il ciclo `for` con uno o più dei seguenti frammenti di codice. (Per ogni alternativa rispondere SI/NO e fornire una motivazione).

```
1) for (i=0; i<(int)size; i++) {  
    kprintf("%c", p[i]);  
}  
2) kprintf("%s", p);  
3) kprintf("%s", &p);  
4) file_write(fd, p, size);
```

SOLUZIONI

Domanda 1 (Paginazione, Working Set e Page Fault)

A) Calcolo Pagine per la Matrice M:

Essendo `sizeof(data_t) = 1024 Byte`, la dimensione totale della matrice $M[64][32]$ è:

$$\text{Dimensione M} = (64 \times 32) \text{ elementi} \times 1024 \text{ B} = 2.048 \times 1024 = 2.097.152 \text{ Byte.}$$

Sapendo che la dimensione della pagina è $2 \text{ KB} = 2048 \text{ Byte}$, e che l'indirizzo di partenza `0x5820C800` è allineato alla pagina (multiplo di 2048), il numero esatto di pagine è:

$$\text{Numero Pagine} = \frac{2.097.152}{2048} = \mathbf{1024 \text{ pagine.}}$$

B) Riferimenti Totali di Memoria (NT):

Il ciclo k non si moltiplica per 5, bensì i valori ammessi per j variano in base al valore di k . Il numero totale di esecuzioni del ciclo interno è:

$$\text{Iterazioni} = 64 \times \sum_{k=0}^4 (\text{iterazioni di } j)$$

Per $k \in \{0, 1, 2, 3\}$, j itera 8 volte (fino a $j = 28$ o $29, 30, 31$). Per $k = 4$, j inizia da 4 e itera 7 volte (fino a $j = 28$). Totale iterazioni $j = 8 + 8 + 8 + 8 + 7 = 39$.

$$\text{Totale Esecuzioni} = 64 \times 39 = \mathbf{2496.}$$

Ad ogni iterazione avviene solo una scrittura in memoria ($NW = 1$). N è una macro del preprocessore, quindi non genera una lettura NR . Pertanto $NT = 2496 + 0 = \mathbf{2496 \text{ riferimenti.}}$

C) Calcolo Page Fault Generati:

Ogni pagina contiene $2048/1024 = \mathbf{2 \text{ elementi}}$. L'incremento $j+ = 4$ nel ciclo più interno forza un accesso a elementi distanti tra loro di $4 \times 1024 = 4096 \text{ Byte}$, scavalcando quindi sempre 2 pagine a ogni iterazione. Per comprendere l'effetto della politica Working Set (con $\Delta = 10$), analizziamo il passaggio tra la prima esecuzione del ciclo più esterno ($k = 0$) e la seconda ($k = 1$):

- **Disposizione in Memoria:** Poiché una pagina tiene 2 elementi, dati consecutivi come $M[0][0]$ e $M[0][1]$ condividono la stessa pagina fisica (Pagina 0).
- **Accessi al passo k=0:** All'inizio ($k = 0$), l'accesso a $M[0][0]$ causa il primo Page Fault e carica la Pagina 0. Il programma esegue poi l'intero ciclo per $k = 0$. In questa fase, il "contatore temporale" degli accessi avanza di: $64 (\text{iterazioni di } i) \times 8 (\text{iterazioni di } j) = \mathbf{512 \text{ accessi a memoria.}}$
- **Accesso al passo k=1:** Incrementato k , il programma tenta di accedere a $M[0][1]$. Pur trovandosi anch'esso nella Pagina 0, l'ultimo riferimento a questa pagina risale esattamente a 512 accessi fa (la lettura di $M[0][0]$).
- **Sfratto (Eviction):** La politica del Working Set impone un limite di sopravvivenza rigoroso. Essendo $512 \gg \Delta$ (dove $\Delta = 10$), la Pagina 0 è "scaduta" da molto tempo e il frame fisico è già stato liberato dal sistema. L'accesso a $M[0][1]$ genererà pertanto un nuovo Page Fault.

Questo comportamento si ripete sistematicamente per l'intera matrice. Conseguenza: assenza totale di località temporale e spaziale utile. **Tutti i 2496 accessi a memoria provocheranno un Page Fault.**

D) Completamento Codice e Ottimizzazione:

Il frammento va completato traducendo le coordinate bidimensionali (i, j) in indice lineare k per matrici row-major: $i = k/N$ e $j = k \% N$.

```
for (k = 0; k < 2*N*N; k++) {
    V[k].diag = (k / N) - (k % N);
}
```

Calcolo Page Fault ottimizzato: Scandendo linearmente l'array, la località spaziale è massimizzata. Poiché una pagina contiene 2 elementi, il primo accesso caricherà la pagina (1 Fault), e il secondo accesso all'elemento adiacente troverà la pagina già in memoria (1 Hit, poiché l'ultimo accesso è avvenuto 1 istante fa, che rientra abbondantemente nel $\Delta = 10$). Questo genera **1 Fault ogni 2 elementi**. Su 2048 elementi totali, il programma produrrà il numero minimo teorico possibile: **1024 Page Fault**.

Domanda 2 (File System Unix-like e Inode)

Dati preliminari dell'architettura:

Dimensione blocco = 4 KB = 2^{12} Byte.

Dimensione puntatore = 32 bit = 4 Byte.

Puntatori per blocco = $4096/4 = 1024 = 2^{10}$.

Capacità Disco = 2 TB = 2^{41} Byte.

A) Calcolo limiti superiori per N_2 e N_3 :

L'obiettivo è calcolare il numero massimo di file (N) compatibili con i vincoli del file system.

• Calcolo N_2 (Indiretto Doppio):

Un file inizia a usare l'indiretto doppio quando supera la capacità dei puntatori diretti e del singolo indiretto: Soglia = $(12 + 1024) \times 4 \text{ KB} = 4144 \text{ KB} \approx 4,1 \text{ MB}$.

Poiché il file system contiene solo file di dimensione compresa tra 8 MB e 8 GB, *tutti i file presenti richiedono l'indiretto doppio* ($8 \text{ MB} > 4,1 \text{ MB}$). Per massimizzare il numero di file (N_2), ipotizziamo che tutti i file abbiano la dimensione minima consentita (8 MB):

$$N_2 = \frac{\text{Spazio Disco}}{\text{Dimensione Minima}} = \frac{2 \text{ TB}}{8 \text{ MB}} = \frac{2^{41} \text{ Byte}}{2^{23} \text{ Byte}} = 2^{18} = \mathbf{262.144 \text{ file}}$$

• Calcolo N_3 (Indiretto Triplo):

Un file necessita dell'indiretto triplo quando la sua dimensione supera il doppio indiretto. Soglia = $(12 + 1024 + 1024^2) \times 4 \text{ KB} = 4.198.448 \text{ KB}$.

Per far scattare l'allocazione del triplo indiretto, basta aggiungere 1 Byte a questa soglia. Per massimizzare N_3 , dobbiamo riempire il disco di file di questa dimensione limite (assimilabile a circa 4 GB):

$$N_3 = \lfloor \frac{2 \text{ TB}}{4.198.448 \text{ KB}} \rfloor = \lfloor \frac{2.147.483.648 \text{ KB}}{4.198.448 \text{ KB}} \rfloor = \mathbf{511 \text{ file}}$$

B) Calcolo Blocchi e Frammentazione per un file da 20.490,5 KB:

Si necessitano $\lceil 20490,5/4 \rceil = \mathbf{5123 \text{ blocchi dati}}$. Questi vengono mappati come segue:

- **12 blocchi** dai puntatori diretti.
- **1024 blocchi** dal singolo indiretto.
- I restanti **4087 blocchi** dal doppio indiretto.

Frammentazione interna:

I 5123 blocchi dati allocati fisicamente offrono uno spazio di $5123 \times 4 \text{ KB} = 20.492 \text{ KB}$.

Frammentazione = Spazio Allocato – Spazio Utilizzato = $20.492 \text{ KB} - 20.490,5 \text{ KB} = \mathbf{1,5 \text{ KB (1536 Byte)}}$

Calcolo Blocchi Indice:

I 12 puntatori diretti risiedono nella struttura i-node e non consumano blocchi aggiuntivi.

Serve **1 blocco** indice per il singolo indiretto.

Servono $\lceil 4087/1024 \rceil = 4$ blocchi indice di base per mappare i 4087 blocchi finali, più **1 blocco** indice radice per puntare a questi 4. In totale, il doppio indiretto consuma $1 + 4 = 5$ blocchi.

$$\text{Totale Blocchi Indice} = 0 + 1 + 5 = \mathbf{6 \text{ blocchi}}$$

C) Posizionamento con lseek:

L'istruzione `lseek(fd, -220, SEEK_END)` sposta il cursore di lettura/scrittura di esattamente 1 MB all'indietro rispetto alla fine del file.

- Dimensione file in Byte: $20.490,5 \text{ KB} \times 1024 = 20.982.272 \text{ Byte}$.
- Valore Offset da sottrarre: $2^{20} = 1.048.576 \text{ Byte}$.
- Nuova posizione assoluta in Byte: $20.982.272 - 1.048.576 = 19.933.696 \text{ Byte}$.

Sapendo che i blocchi logici sono numerati a partire da 0 e misurano 4096 Byte, calcoliamo l'indice del blocco dividendo la posizione assoluta per la dimensione del blocco e troncando il risultato:

$$\text{Indice Blocco Logico} = \lfloor \frac{19.933.696}{4096} \rfloor = \lfloor 4866,625 \rfloor = \mathbf{4866}$$

(Nota: Questo corrisponde al 4867-esimo blocco logico del file).

Domanda 3 (Prestazioni e Allocazione File)

Premessa Teorica sui Costi di Accesso:

Le prestazioni degli algoritmi sui file dipendono dal costo dell'**Accesso Casuale** (Random Access) ai blocchi del file system:

- **Contigua e Inodes:** Costo di accesso al k -esimo blocco pari a $O(1)$, grazie al calcolo diretto dell'offset o all'attraversamento di un albero a profondità limitata e costante.
- **Linked List e FAT:** Costo di accesso al k -esimo blocco pari a $O(N)$. Per raggiungere un offset casuale è necessario scorrere linearmente i puntatori (sul disco per la lista concatenata, in RAM per la FAT).

A) Complessità per l'ordinamento $O(N \log N)$ sul file:

Un algoritmo di ordinamento sul posto (in-place) effettua continui accessi casuali e scambi (swap) tra record distanti. Poiché il numero di operazioni algoritmiche è $O(N \log N)$, il costo totale dipenderà dal costo di ciascun accesso al disco.

- **Allocazione Contigua e Inodes:** $O(N \log N)$. (Motivazione: Ogni accesso casuale o scambio richiesto dall'algoritmo ha costo $O(1)$. La complessità finale coincide con quella dell'algoritmo).
- **Allocazione Linked list e FAT:** $O(N^2 \log N)$. (Motivazione: Ogni singolo accesso casuale o scambio dei $N \log N$ previsti richiede di scorrere sequenzialmente la lista dei blocchi con costo $O(N)$. Moltiplicando i fattori si ottiene $N \log N \times N$).

B) Complessità per la stampa "Indice e File Relativo":

L'operazione prevede la lettura *sequenziale* del file Indice (costo complessivo $O(N)$ per tutti i file system) e N accessi *casuali* al file Relativo, poiché i record non sono ordinati fisicamente. Il costo dominante è dato dagli N accessi casuali al file Relativo.

- **Allocazione Contigua e Inodes: $O(N)$.** (Motivazione: Vengono effettuati N accessi casuali, ciascuno con costo risolutivo $O(1)$. Il costo totale è dominato da $N \times O(1) = O(N)$).
- **Allocazione Linked list e FAT: $O(N^2)$.** (Motivazione: Vengono effettuati N accessi casuali. A causa della struttura a lista, raggiungere l'indirizzo logico fornito dall'indice richiede in media un tempo proporzionale alla dimensione del file, ovvero $O(N)$. Il costo totale è dato da N accessi $\times$ costo $O(N)$ per accesso).

Domanda 4 (Sincronizzazione OS161 - Producer/Consumer)

A) Posizionamento della struttura condivisa:

La struttura **non può** essere posizionata nello stack di un thread. In OS161, sebbene i thread del kernel condividano lo stesso spazio di indirizzamento, lo stack è rigorosamente **privato per ogni thread**. Allocare dati condivisi sullo stack di un thread specifico è pericoloso: se quel thread termina, lo stack viene deallocato, invalidando i puntatori per gli altri thread. La struttura deve risiedere in un'area di memoria dal ciclo di vita controllato, tipicamente sullo **heap** (allocata dinamicamente tramite `kmalloc`) oppure come **variabile globale** nel segmento dati del kernel.

B) Posizionamento chiamate `lock_create()` e `cv_create()`:

Queste funzioni devono essere chiamate **nella funzione di inizializzazione principale** (es. il "main" del test o la funzione di setup), tassativamente **prima** che i thread producer e consumer vengano creati tramite `thread_fork()`. Se l'allocazione avvenisse all'interno del codice dei producer o del consumer, si genererebbe una gravissima *race condition* in cui più thread tenterebbero di allocare o sovrascrivere i medesimi lock/CV contemporaneamente.

C) Completamento del Codice:

(Nota sulle API: OS161 utilizza `cv_signal` o `cv_broadcast`. Poiché l'architettura proposta usa un'unica Condition Variable per due ruoli diversi — 2 produttori e 1 consumatore — l'uso di `cv_signal` potrebbe causare un problema di lost wakeup se un produttore svegliasse per sbaglio un altro produttore invece del consumatore. Per totale robustezza si utilizza `cv_broadcast`).

```
/* PRODUCER */
void producerWrite (struct prodConsBuf *pc, data_t *dp) {
    lock_acquire(pc->pc_lk);

    /* <1> Attendiamo che il buffer si svuoti.
       Si usa 'while' per la semantica Mesa (ricontrollo post-risveglio) */
    while (pc->dataReady == 1) {
        cv_wait(pc->pc_cv, pc->pc_lk);
    }

    pc->data = *dp;
    pc->dataReady = 1;

    /* <2> Segnaliamo che ci sono dati pronti */
    cv_broadcast(pc->pc_cv); /* o cv_signal(pc->pc_cv); */

    lock_release(pc->pc_lk);
}

/* CONSUMER */
void consumerRead (struct prodConsBuf *pc, data_t *dp) {
```

```

lock_acquire(pc->pc_lk);

/* Attendiamo che il buffer si riempia */
while (pc->dataReady == 0) {
    cv_wait(pc->pc_cv, pc->pc_lk);
}

*dp = pc->data;
pc->dataReady = 0;

/* <3> Segnaliamo ai producer che il buffer è vuoto */
cv_broadcast(pc->pc_cv); /* o cv_signal(pc->pc_cv); */

lock_release(pc->pc_lk);
}

```

Domanda 5 (System Calls: open, close, write in OS161)

A) Allocazione della "per-process file table":

La *per-process file table* (tabella dei file descriptor aperti da un processo) è allocata nella **memoria del kernel**. Essendo una struttura dati critica che contiene puntatori diretti agli oggetti file del sistema operativo (vnode, offset, permessi), deve essere inaccessibile e protetta da manipolazioni arbitrarie da parte del codice utente.

B) Chiamate concorrenti a `write(fd, v, N)` con `fd = 5`:

1. **I processi scrivono sullo stesso file?** **NO**. Il valore 5 è solo un indice locale nella tabella privata di ciascun processo.
2. **Non scrivono mai sullo stesso file?** **NO**. Potrebbero puntare allo stesso file se, ad esempio, i due processi lo hanno aperto esplicitamente lo stesso percorso o se hanno ereditato la tabella dei file in seguito a una `fork()`.
3. **Generalmente file diversi ma a volte lo stesso?** **SI**. È lo scenario standard: le tabelle sono indipendenti, quindi `fd = 5` mapperà strutture diverse, ma la condivisione dello stesso file è possibile tramite ereditarietà o aperture mirate.
4. **Servono fd diversi per lo stesso file?** **NO**. Due processi possono tranquillamente usare lo stesso valore `fd = 5` per riferirsi alla stessa entry condivisa, oppure valori diversi (es. `fd = 3` per P1 e `fd = 5` per P2) che puntano alla medesima struttura file aperta. Il numero assegnato dipende solo dallo storico delle aperture del singolo processo.

C) Sostituzione del ciclo `for` in `sys_write`:

*Analisi delle alternative (assumendo che la dereferenziazione diretta sia tollerata come nell'implementazione originale, senza **copyin**):*

1. `for (i=0; i<(int)size; i++) { kprintf("%c", p[i]); }`
SI. L'istruzione `kprintf` con specificatore di formato `%c` stampa un carattere per volta. Ripetuta per `size` iterazioni, equivale esattamente al comportamento della `putch()`.
2. `kprintf("%s", p);`
NO. Lo specificatore `%s` in C richiede una stringa terminata dal carattere nullo (`\0`). Il buffer passato a una `write` non garantisce la terminazione nulla (è un raw buffer di lunghezza `size`). Questa istruzione ignorerebbe il parametro `size` e continuerebbe a leggere in memoria causando overflow o stampando caratteri illeggibili.

3. `kprintf("%s", &p);`

NO. Grave errore semantico nel linguaggio C. Si sta passando l'indirizzo del puntatore `p` (allocato nello stack locale della funzione kernel) invece del puntatore stesso. Questo porta alla stampa di byte spazzatura dello stack e potenziale crash.

4. `file_write(fd, p, size);`

NO. Il testo specifica che la console non supporta reindirizzamento e bypassa il file system. Il controllo iniziale (`if (fd != STDOUT...)`) conferma che questi descrittori speciali non passano per la `file_write`. In un'architettura simile, invocare `file_write` su fd 1 o 2 fallirebbe in quanto non associati ad alcun `vnode` o struttura *Open File* standard nella file table.